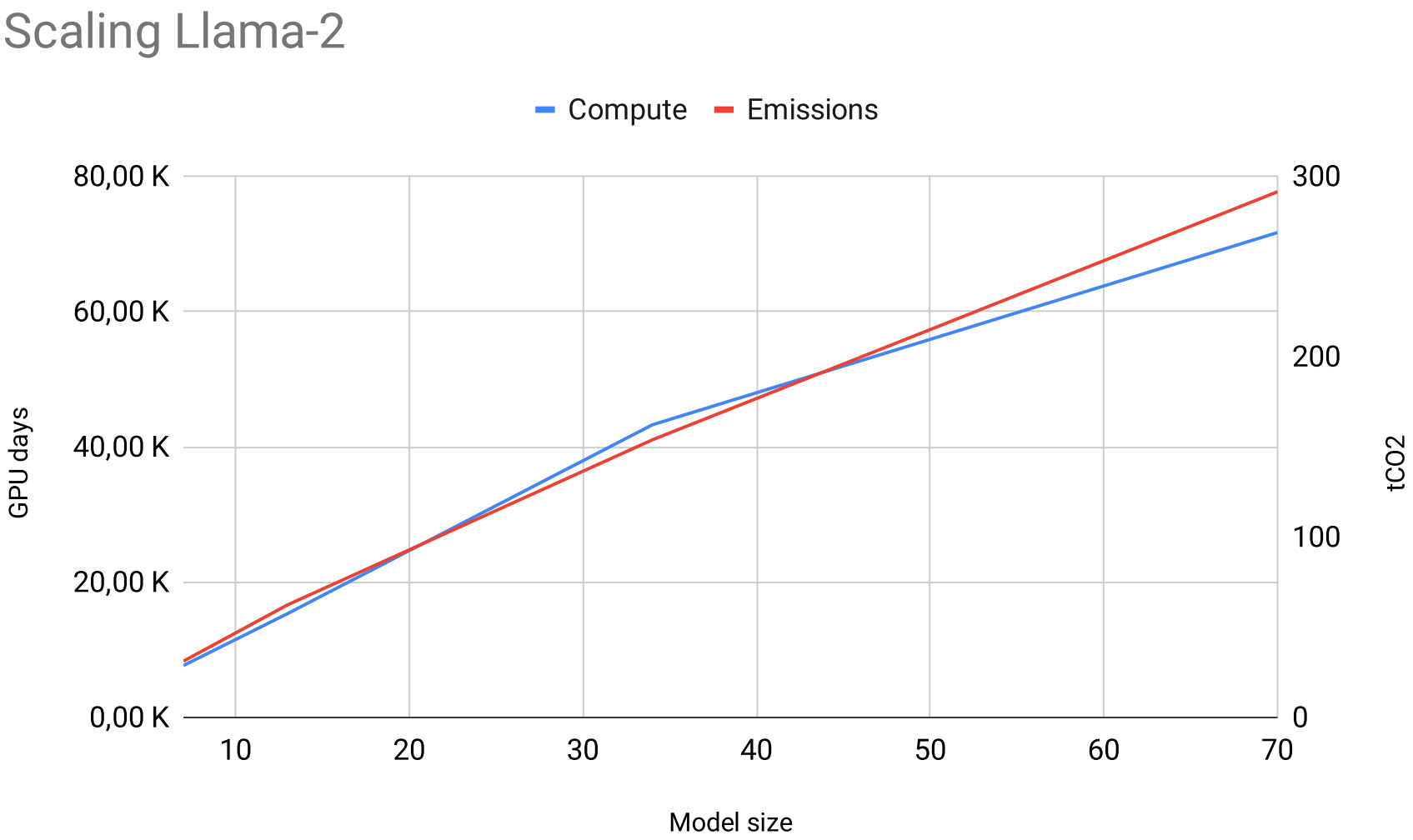
Course 4: Efficient NLP

Introduction

Introduction



Introduction

- Throughput decreases as you increase your model size.
- LMs can become VRAM hungry without proper optimization.

What are the different ways of reducing compute for training/inference and increasing throughput.

Content

1. **Scaling Laws**
2. **Efficient Training**
   a. Mixed precision
   b. Efficient implementations
   c. Multi GPU training
3. **Efficient Fine-Tuning**
   a. Adapters
   b. LoRA
4. **Efficient Inference**
   a. Quantization
   b. KV-Cache
   c. Speculative decoding
5. **Model Reduction**
   a. Distillation
   b. Mixture of experts

Scaling Laws

Scaling Laws

- Scaling Laws for Neural Language Models (Kaplan et al. 2020)

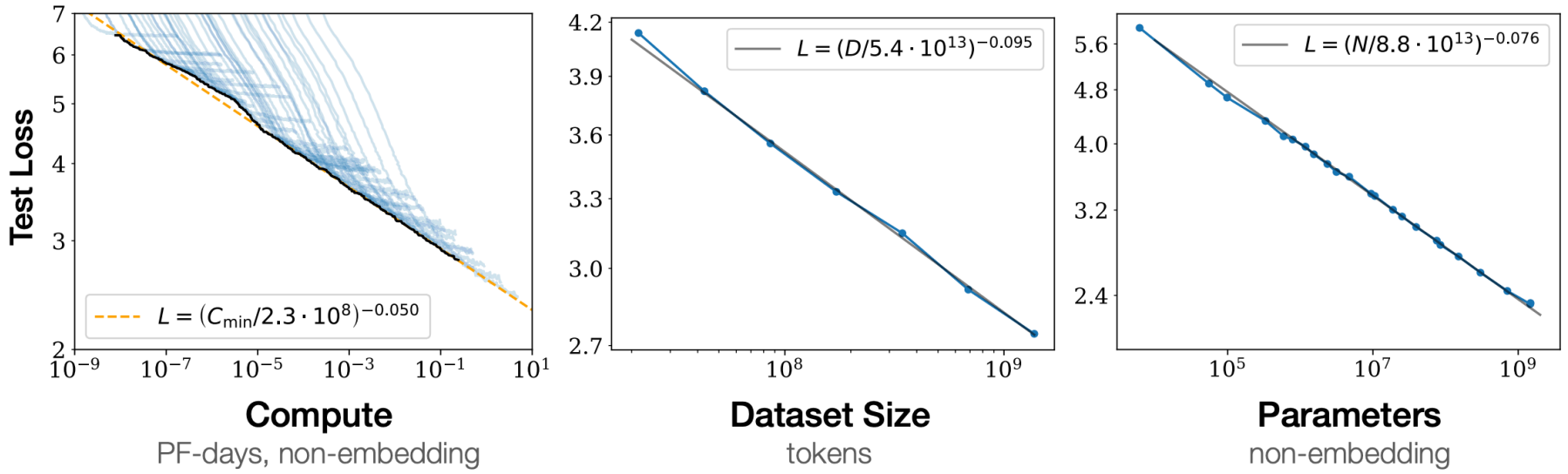
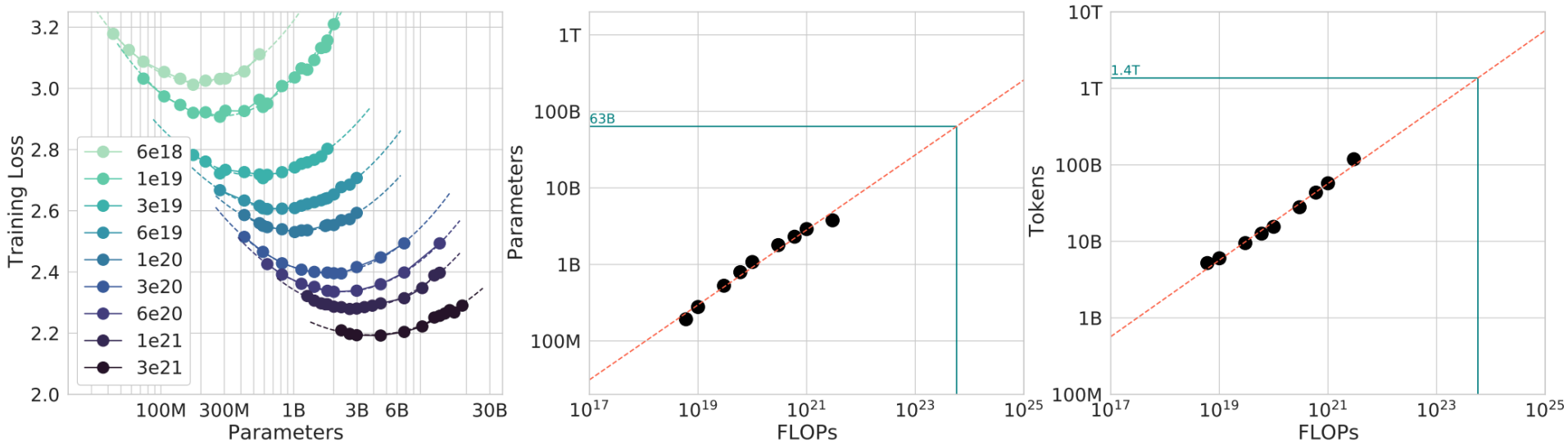


Figure 1 Language modeling performance improves smoothly as we increase the model size, datasetset size, and amount of compute[2] used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Chinchilla Scaling Laws

- Refinement using more data points & better training recipe
- Given C FLOPS, what model size N and training tokens D should one use?



Scaling Laws

Chinchilla Scaling Laws

- Propose a form for the final loss:

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$

- Fit it on data points
  - E = 1.69 ("*entropy of natural language*")
  - A = 406.4, B = 410.7, α = 0.34, β = 0.28

Scaling Laws

Chinchilla Scaling Laws

- Compute $C = O(ND)$ ($C \simeq 6ND$)
- For a given compute level C, there exist an optimal N^* and D^*
  - Training a bigger model on less data => worse
  - Training a smaller model on more data => worse

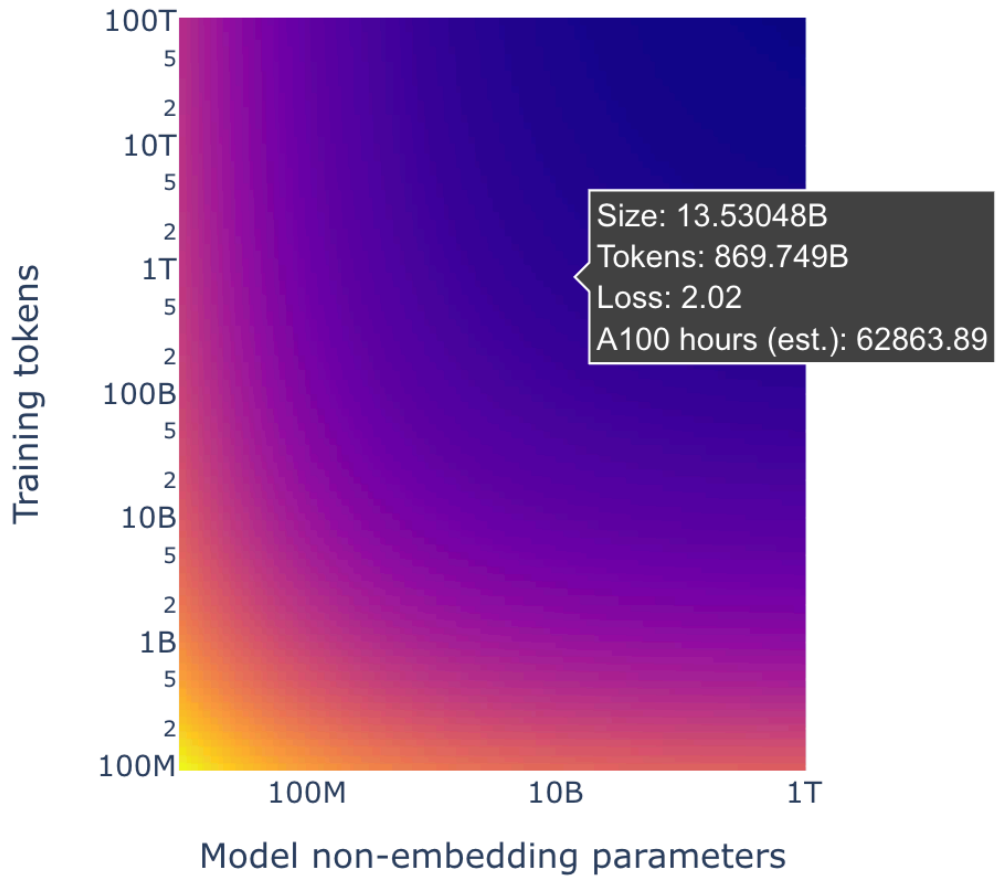
Scaling Laws

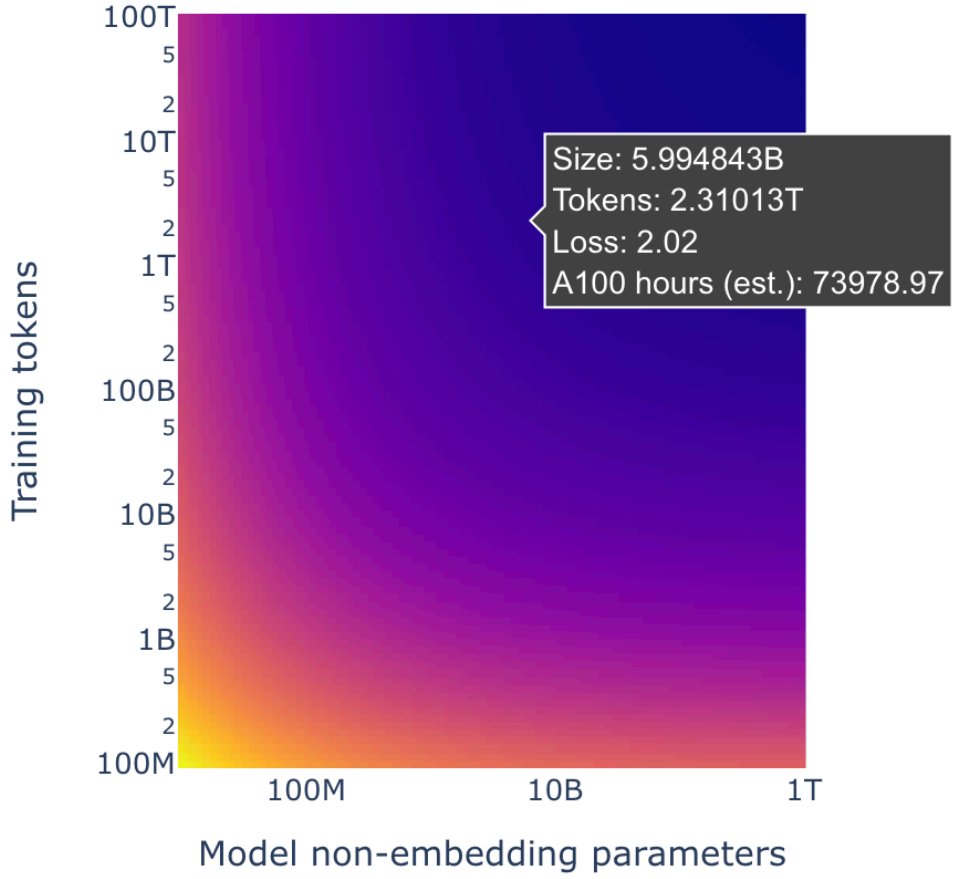
Chinchilla Scaling Laws

- Train for longer than announced by laws
  - Why?
- Over-train smaller models
  - Trade training compute for inference compute
- Example: Mistral-7B model

Scaling Laws

Chinchilla Scaling Laws - In practice



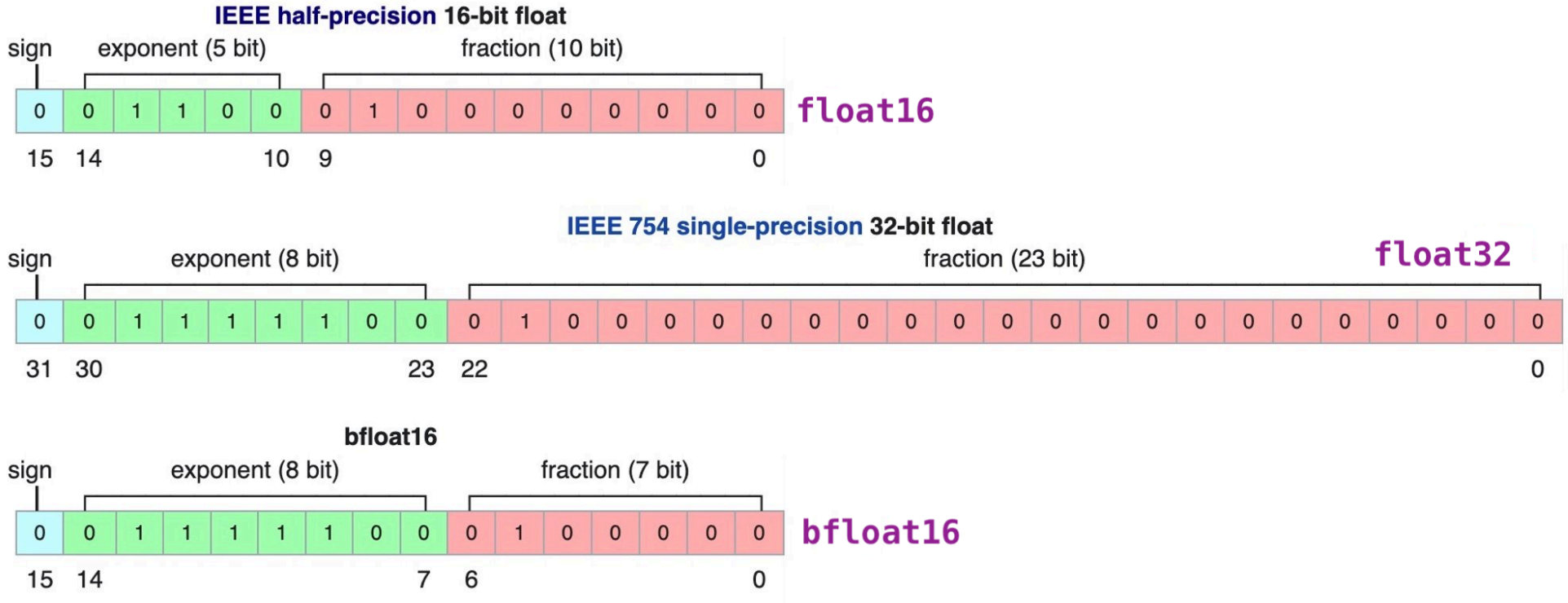


Efficient Training

Mixed precision

- Batch size matters:
  - No bigger than 1-2M tokens (Kaplan et al., 2020)
  - Maximize parallel computation
- Float precision:
  - `float16`: reduces memory usage, good with V100-gen GPUs
  - `bfloat16`: more stability, but only usable with A100-gen GPUs

Mixed precision



Efficient implementations

- xFormers & Memory-efficient attention (Rabe et al. 2021)
  - Classical implementation

$$s_i = \mathrm{dot}(q, k_i), \qquad s'_i = \frac{e^{s_i}}{\sum_j e^{s_j}}, \qquad \mathrm{attention}(q, k, v) = \sum_i v_i s'_i.$$

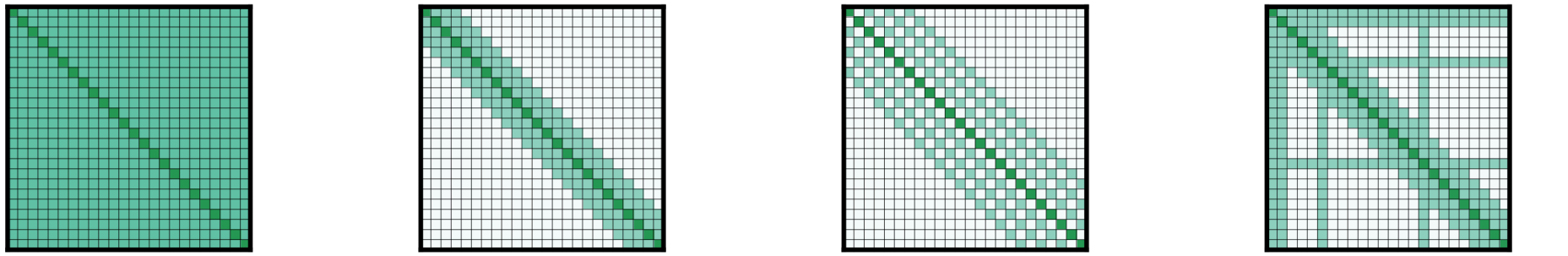
  - Memory-efficient implementation

$$s_i = \mathrm{dot}(q, k_i), \qquad s'_i = e^{s_i}, \qquad \mathrm{attention}(q, k, v) = \frac{\sum_i v_i s'_i}{\sum_j s'_j}.$$

- ~SOTA on V100-gen GPUs

Efficient implementations

- Linear attention (e.g. Beltagy et al. 2020)



(a) Full n^2 attention (b) Sliding window attention (c) Dilated sliding window (d) Global+sliding window

Figure 2: Comparing the full self-attention pattern and the configuration of attention patterns in our Longformer.

- Can be used to adapt model for efficient inference
- Used in Mistral-7B

Multi GPU training

- Dream scenario:
  - Model fits on the GPU
  - Forward + backward fit with the `batch_size`
  - Optimization fits memory

Multi GPU training

- Optimization OOM scenario
  - Model fits on the GPU
  - Forward + backward fit with the `batch_size`
  - Optimization saturates GPU
- Use memory-efficient optimizers
  - Adafactor: factor momentum matrix in Adam
  - CAME: regularize Adafactor
  - LION: only tracks momentum

Multi GPU training

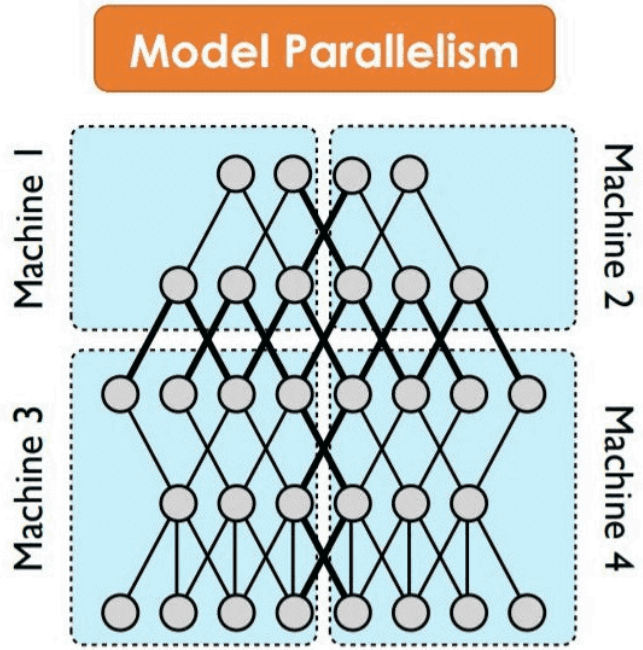
- Forward/backward OOM scenario
  - Model fits on the GPU
  - Forward + backward saturates with the `batch_size`
- Use gradient accumulation
  - Compute forwards with `micro_batch_size` $\ll$ `batch_size`
  - Sum `micro_batch_size//batch_size` gradients
  - Backward once

Multi GPU training

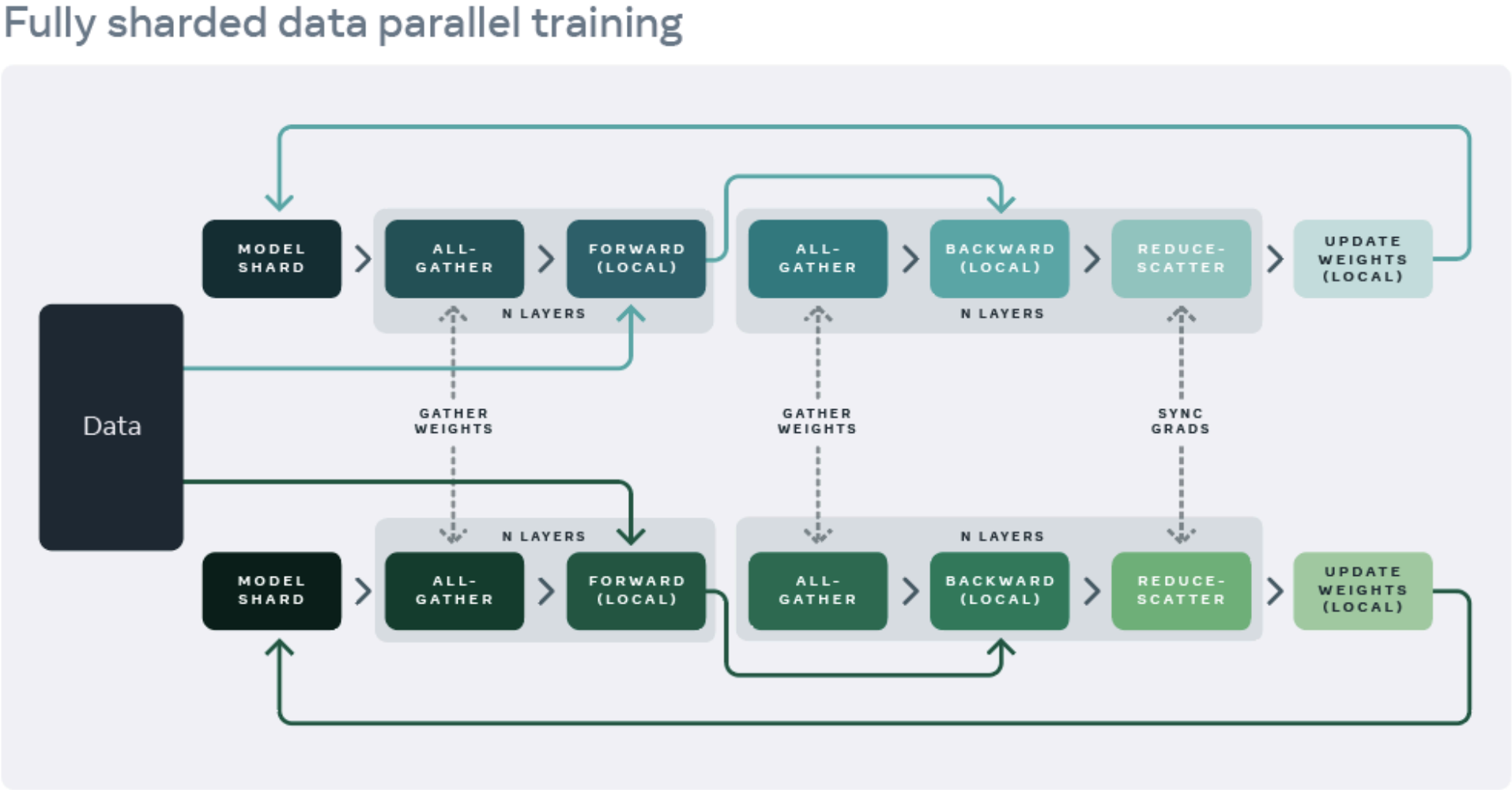
- **Distirbuted Data Parallel (DDP)** with k GPUs
  - Copy the model on the k GPUs
  - Send a micro-batch to each GPU
  - Compute forward/backward in parallel on each GPU
  - *Send* gradients to one GPU & optimize
  - *Send* weight updates to each GPU

Multi GPU training

- Model OOM scenario
  - Model does not fit on one GPU (e.g. `micro_batch_size=1` fails)
- Model parallelism



Multi GPU training



Multi GPU training

DeepSpeed

- Similar to FSDP:
  - Shares model weights...
  - but also optimizer states...
  - and gradients
- For relevant sizes: not that different in speed ⏳

Efficient Fine-Tuning

Adapters

Parameter-Efficient Transfer Learning for NLP (Houlsby et al. 2019)

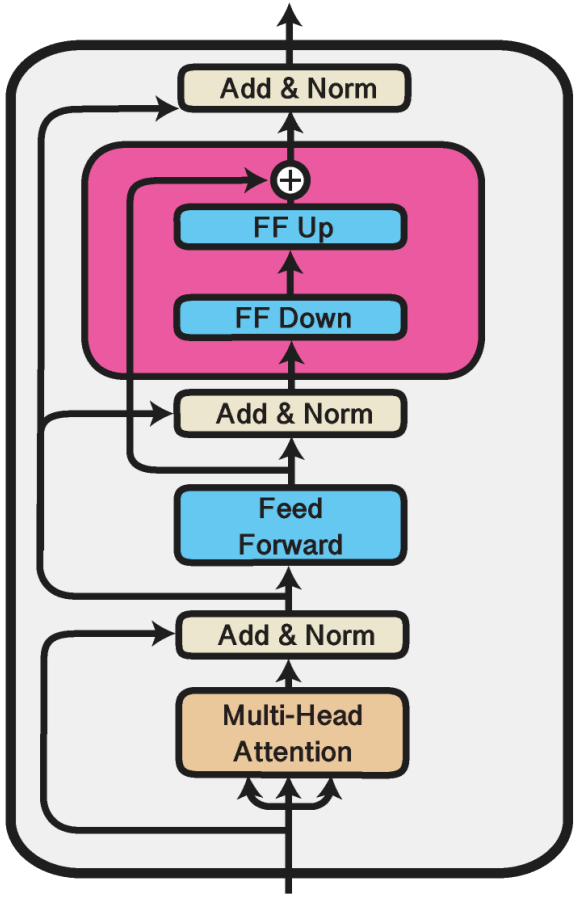
The Problem: Fine-tuning all parameters of a pre-trained model is expensive and requires storing a complete copy for each task.

The Solution: Insert small, trainable modules (adapters) into the frozen pre-trained model.

Each adapter module contains:

- A down-projection: $d_{model} \rightarrow d_{bottleneck}$
- A non-linearity (typically ReLU or GELU)
- An up-projection: $d_{bottleneck} \rightarrow d_{model}$
- A residual connection

Adapters


Add & Norm
FF Up
FF Down
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention

Adapters

For an input h of dimension d_{model}:

$$\text{Adapter}(h) = h + W_{up} \cdot \sigma(W_{down} \cdot h + b_{down}) + b_{up}$$

Where:

- $W_{down} \in \mathbb{R}^{d_{bottleneck} \times d_{model}}$ reduces dimensionality
- $W_{up} \in \mathbb{R}^{d_{model} \times d_{bottleneck}}$ restores dimensionality
- σ is a non-linear activation
- The residual connection preserves the original representation

Adapters

Consider $d_{model} = 768$ (BERT-base) and $d_{bottleneck} = 64$:

Parameters per adapter:

- Down-projection: $768 \times 64 = 49,152$
- Up-projection: $64 \times 768 = 49,152$
- Biases: $64 + 768 = 832$
- **Total**: ~99K parameters

Original transformer layer: ~7M parameters

Reduction: Only ~1.4% of parameters are trainable per layer!

Adapters

Intuition: Pre-trained representations capture general language understanding. The adapter layers learn task-specific transformations.

The bottleneck forces the model to learn a compressed, task-relevant representation. This acts as a form of regularization and prevents overfitting on small datasets.

Residual connections ensure that if no adaptation is needed, the adapter can learn to output near-zero values.

LoRA

Low-Rank Adaptation of Large Language Models (Hu et al. 2021)

Adapters add sequential operations (down-project -> activate -> up-project), which increases inference latency.

Key observation: Weight updates during fine-tuning often have low intrinsic rank. We can represent these updates as low-rank decompositions.

LoRA

For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, represent the update as:

$$W = W_0 + \Delta W = W_0 + BA$$

Where:

- $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$
- $r \ll \min(d, k)$ is the rank
- W_0 remains frozen
- Only A and B are trained

LoRA

During training:

$$h = W_0 x + \frac{\alpha}{r} BAx$$

The scaling factor $\frac{\alpha}{r}$ (where α is a constant, typically $\alpha = r$) controls the magnitude of the LoRA update relative to the pre-trained weights.

During inference: Merge BA into W_0 for zero additional latency:

$$W_{merged} = W_0 + BA$$

LoRA

Consider adapting a query projection in attention where $d = k = 1024$ and rank $r = 8$:

Full fine-tuning: $1024 \times 1024 = 1,048,576$ parameters

LoRA:

- Matrix A: $8 \times 1024 = 8,192$ parameters
- Matrix B: $1024 \times 8 = 8,192$ parameters
- **Total**: 16,384 parameters

98% fewer parameters!

LoRA

Which Layers to Adapt?

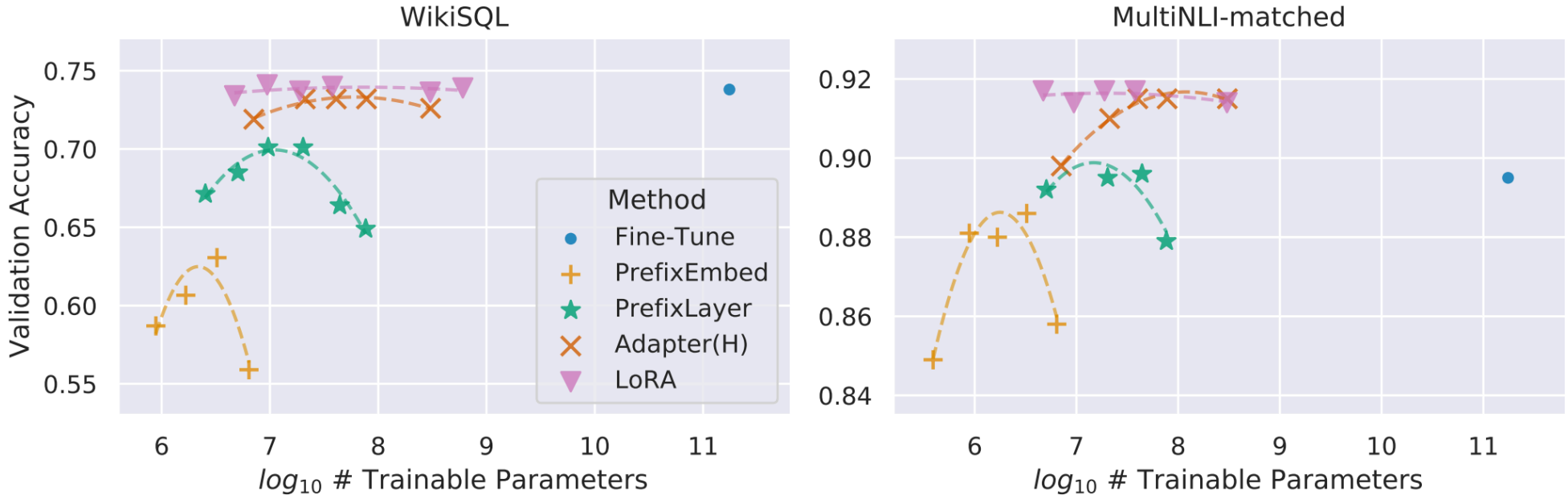
Attention matrices (Q, K, V, O projections): Most commonly adapted. Different attention heads may need different adjustments for new tasks.

Feed-forward layers: Can also be adapted, though often less critical.

Embedding layers: Rarely adapted with LoRA.

Empirically, adapting just the attention query and value projections often suffices for good performance.

LoRA


WikiSQL
MultiNLI-matched
Validation Accuracy
0.75
0.70
0.65
0.60
0.55
0.92
0.90
0.88
0.86
0.84
6
7
8
9
10
11
log10 # Trainable Parameters
Method
Fine-Tune
PrefixEmbed
PrefixLayer
Adapter(H)
LoRA

Efficient inference

Previous methods hold

- Efficient attention implementations & variants
  - xFormers
  - Linear attention
- Model parallelism (FSDP & DeepSpeed)
- LORA weights for fast model "switching"
  - Keep big model in memory
  - Load task-specific LoRA when required

Quantization

Modern LLMs use `float16` or `bfloat16` representations:

- 2 bytes per parameter
- A 7B parameter model requires ~14GB just for weights
- Add activations, optimizer states -> 40GB+ during inference

Goal: Reduce precision to `int8` (1 byte) or `int4` (0.5 bytes) while maintaining accuracy.

Quantization

The challenge: Map continuous floating-point values to discrete integers.

For a weight $w \in [-w_{max}, w_{max}]$ and target quantization to b bits:

$$w_q = \text{round}\left(\frac{w}{s}\right) \in [-2^{b-1} - 1, 2^{b-1} - 1]$$

Where s is the scale factor. But how do we determine s?

Quantization

Symmetric quantization: Assumes weights are centered around zero.

$$s = \frac{\max(|w|)}{2^{b-1} - 1}$$

For 4-bit quantization ($b = 4$), the range is $[-7, 7]$ (using signed integers).

Example: If $\max(|w|) = 2.3$, then:

$$s = \frac{2.3}{7} \approx 0.329$$

Quantization

Consider weights: $x = [-2.3, -1.0, 0.5, 1.8]$

Step 1 - Find scale:

$$s = \frac{\max(|-2.3|, |-1.0|, |0.5|, |1.8|)}{7} = \frac{2.3}{7} = 0.329$$

Step 2 - Quantize:

$$q = \text{round}(x/s) = \text{round}\left(\frac{[-2.3, -1.0, 0.5, 1.8]}{0.329}\right)$$

$$q = \text{round}([-6.99, -3.04, 1.52, 5.47]) = [-7, -3, 2, 6]$$

Quantization

Step 3 - Dequantize (for verification):

$$\hat{x} = s \cdot q = 0.329 \times [-7, -3, 2, 6]$$

$$\hat{x} = [-2.30, -0.99, 0.66, 1.97]$$

Quantization error:

$$|x - \hat{x}| = [0.00, 0.01, 0.16, 0.17]$$

The maximum absolute error is 0.17, which is reasonable for 4-bit precision.

Quantization

GPTQ (Frantar et al. 2023)

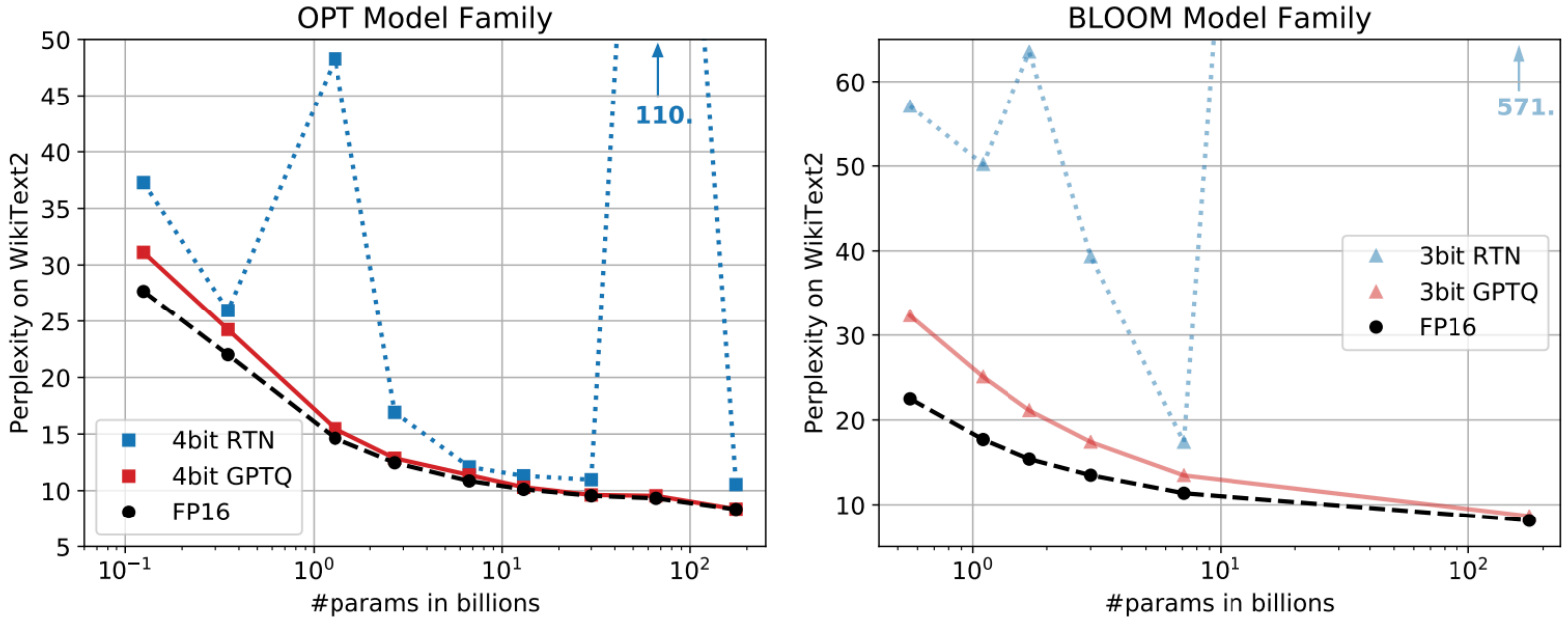


Figure 1: Quantizing OPT models to 4 and BLOOM models to 3 bit precision, comparing GPTQ with the FP16 baseline and round-to-nearest (RTN) (Yao et al., 2022; Dettmers et al., 2022).

Quantization

GPTQ: The Algorithm

Key insight: Quantize weights while minimizing impact on layer outputs, not just weight error.

For each layer ℓ:

1. Collect input activations X from calibration data
2. Compute original outputs: $Y = WX$
3. Quantize W to W_q minimizing $||WX - W_qX||^2$

This is a **one-time** process done after training.

Quantization

GPTQ: Computational Cost

OPT	13B	30B	66B	175B
Runtime	20.9m	44.9m	1.6h	4.2h
BLOOM	1.7B	3B	7.1B	176B
Runtime	2.9m	5.2m	10.0m	3.8h

Table 2: GPTQ runtime for full quantization of the 4 largest OPT and BLOOM models.

Quantization is fast: minutes to hours on a single A100 for 7-70B models.

Quantization

GPTQ: Performance Gains

GPU	FP16	3bit	Speedup	GPU reduction
A6000 – 48GB	589ms	130ms	4.53×	8 → 2
A100 – 80GB	230ms	71ms	3.24×	5 → 1

Table 6: Average per-token latency (batch size 1) when generating sequences of length 128.

Memory reduction: ~4× for 4-bit quantization

Speed improvement: 2-3× due to reduced memory bandwidth

Quantization

GPTQ: Quality Preservation

OPT	Bits	125M	350M	1.3B	2.7B	6.7B	13B	30B	66B	175B
full	16	27.65	22.00	14.63	12.47	10.86	10.13	9.56	9.34	8.34
RTN	4	37.28	25.94	48.17	16.92	12.10	11.32	10.98	110	10.54
GPTQ	4	**31.12**	**24.24**	**15.47**	**12.87**	**11.39**	**10.31**	**9.63**	**9.55**	**8.37**

Perplexity degradation is minimal for 4-bit quantization on large models. Smaller models are more sensitive to quantization.

KV-cache

When generating text, transformers compute attention over all previous tokens at each step:

Step 1: Generate token 1 from prompt
Step 2: Generate token 2 attending to prompt + token 1
Step 3: Generate token 3 attending to prompt + token 1 + token 2
...

Each step recomputes attention for all previous tokens. This is wasteful.

KV-cache

Recall that attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

For the i-th token generation:

- Q_i: query for the current token ($1 \times d_{model}$)
- $K_{1:i}$: keys for all previous tokens ($i \times d_{model}$)
- $V_{1:i}$: values for all previous tokens ($i \times d_{model}$)

KV-cache

The keys and values for previous tokens **never change** during generation. Once computed, they remain constant.

Without caching: Recompute K and V for all previous tokens at each step.

With caching: Store computed K and V and only compute for new tokens.

KV-cache

Example: Predicting token 4

At step 3, we already have:

$$K_{\text{cache}} = [x_1 W_K, x_2 W_K, x_3 W_K], \quad V_{\text{cache}} = [x_1 W_V, x_2 W_V, x_3 W_V].$$

When decoding token 4:

$$Q_4 = x_4 W_Q,$$

$$\text{Attention}(x_4) = \text{softmax}\left(\frac{Q_4 K_{\text{cache}}^\top}{\sqrt{d_k}}\right) V_{\text{cache}}.$$

Only Q_4 is new, K, V come from the cache.

KV-cache

Why Not Cache (Q)?

- (Q_t) is **query-specific**: used once to compute the output for token t.
- Future steps never reuse old queries.
- K, V form the **memory** of past tokens;
- Q_t expresses **what the current token wants to retrieve** from that memory.

$$\text{At step } t: \quad \text{use } Q_t \text{ vs. cached } (K_{1:t-1}, V_{1:t-1})$$

KV-Cache

Without cache for sequence length n:

- Compute K and V: $O(n \cdot d_{model}^2)$ per step
- Total for m generated tokens: $O(m \cdot n \cdot d_{model}^2)$

With cache:

- Compute only new K_i and V_i: $O(d_{model}^2)$ per step
- Total: $O(m \cdot d_{model}^2)$

Speedup: Approximately $n\times$ faster (where n is prompt + generated length).

KV-Cache

✅ Dramatic speedup for long context generation
✅ Straightforward implementation

❌ Memory grows linearly with sequence length
❌ Limits batch size (memory constraint)
❌ Cannot be used with attention variants that recompute (e.g., some sparse attention)

Speculative decoding

An **LLM** can **predict multiple tokens in a single forward pass** :

- **Speculative decoding** [5] allows an LLM to **"guess" future tokens** while generating current tokens, **all within a single forward pass**.
- By running a draft model to predict multiple tokens, the main model (larger) only has to verify the predicted tokens for "correctness".

Speculative decoding

1. **Prefix**: [BOS]
2. **Assistant**: [BOS] The quick brown sock jumps
3. **Main**: [BOS] The quick brown fox / sock jumps
4. **Assistant**: [BOS] The quick brown fox jumps over the crazy dog
5. **Main**: The quick brown jumps over the lazy / crazy dog
6. ...

Speculative decoding

The main model just verifies that the distribution $q(x)$, computed by the assistant is not too far from the distribution $p(x)$ it computes within a forward pass.

The expected number of tokens generated within one looop of speculative decoding can be theorithically formulated as:

$$E(\#generated_tokens) = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

Which is the forward passes' reduction factor.

Speculative decoding

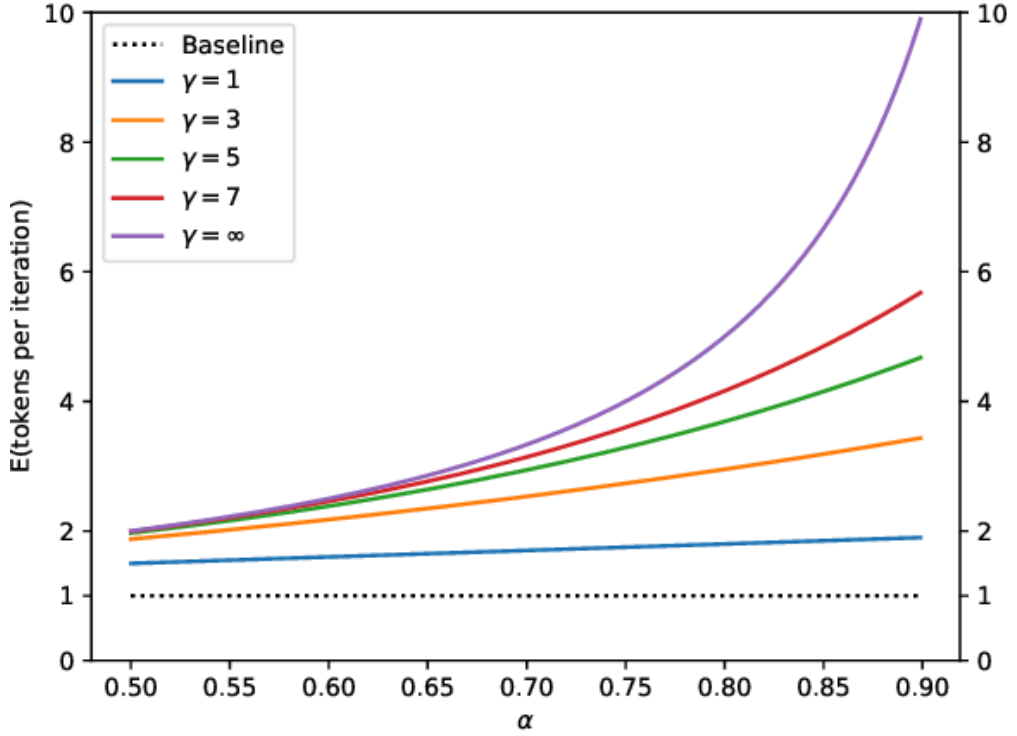
α **(acceptance rate)**: Probability that draft model's token matches large model's preference.

- Depends on how similar draft and large models are
- Typical values: 0.5 - 0.8

γ **(draft length)**: Number of tokens proposed by draft model.

- Limited by verification cost (memory/compute)
- Typical values: 3 - 10

Speculative decoding



The expected number of tokens generated via speculative decoding as a function of α for various values of γ.

Speculative decoding

In order **to take the most out of speculative decoding**, the distance between $q(x)$ **and** $p(x)$ **needs to be minimal**.

How to reduce the distance between $q(x)$ and $p(x)$ when the assistance model is smaller?

- Quantization
- Distillation
- Over-training on the same dataset as the main model

Model Reduction

Distillation

Goal: Transfer knowledge from a large, accurate "teacher" model to a smaller, faster "student" model.

Why? Pre-training from scratch is expensive. Distillation allows creating smaller models that retain much of the teacher's performance at a fraction of the cost.

First proposed by Hinton et al. (2015), refined for NLP by Sanh et al. (2019) with DistilBERT.

Distillation

Standard training uses "hard" labels: correct token = 1.0, all others = 0.0

⚠️ We **lose information** about the **model's uncertainty** and token similarities.

Example: For "I'm going to the ___"

- Teacher might give: zoo (0.7), park (0.15), doctor (0.1), ...
- Hard label gives: zoo (1.0), all others (0.0)

The teacher's distribution encodes valuable information about plausible alternatives.

Distillation: Soft Targets

Temperature-scaled softmax: Control the "softness" of probability distributions:

$$p_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

Where T is the temperature.

$\tau = 1$: Standard softmax (sharp distribution)
$\tau > 1$: Softer distribution (more uniform)
$\tau \to \infty$: Uniform distribution

Distillation

Student training objective combines two losses:

$$\mathcal{L} = \alpha \mathcal{L}_{distill} + (1 - \alpha)\mathcal{L}_{CE}$$

Distillation loss: Match teacher's soft targets

$$\mathcal{L}_{distill} = \tau^2 \cdot \mathrm{KL}(P_{teacher}(T)||P_{student}(T))$$

Cross-entropy loss: Fit hard labels

$$\mathcal{L}_{CE} = -\sum_i y_i \log p_{student}(i)$$

Distillation

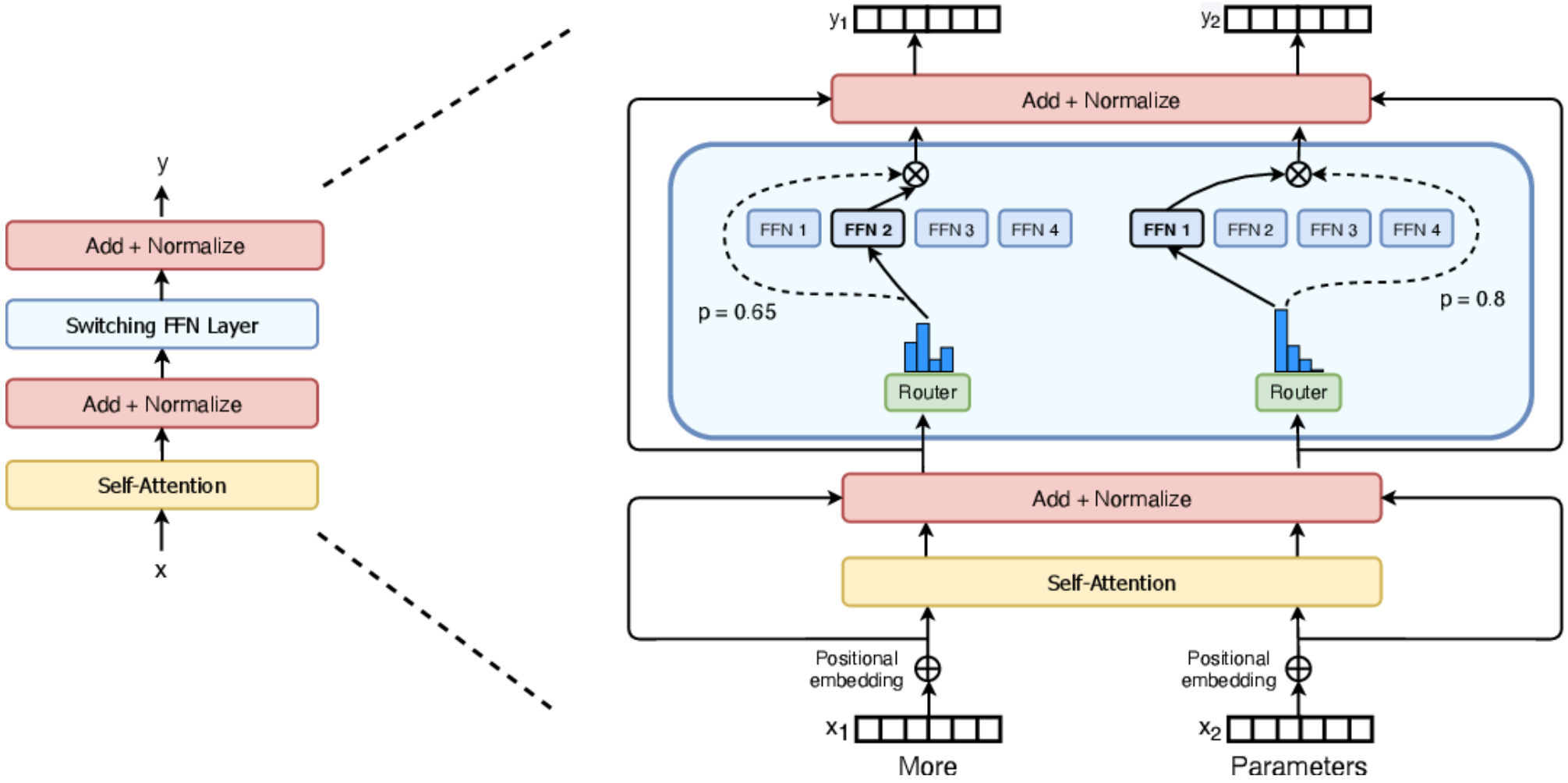
Why τ^2?

The τ^2 factor **compensates for the gradient magnitude** when using temperature-scaled softmax.

Derivation intuition: The gradient of KL divergence with temperature τ scales as $\frac{1}{\tau}$, so we multiply by τ^2 to maintain gradient scale comparable to the CE term.

In practice, τ is typically 2-4, and α is around 0.5-0.9 (favoring distillation).

Mixture of experts



Mixture of experts

- Reduced computation during training and inference since we only need to run $1/N$th of the FFN weights.
- Unstable during training: can struggle to generalize, thus prone to overfitting.
- Load balancing is crucial: we do not want a subset of experts to be under-utilized.

Mixture of experts

A learned gating network G decides which experts E to send a part of the input:

$$y = \sum_{i=1}^{n} G(x)_i \times E_i(x)$$

Where $G(x)_i$ denotes the n-dimensional output of the gating network for the i-th expert, and $E_i(x)$ is the output of the i-th expert network

Mixture of experts

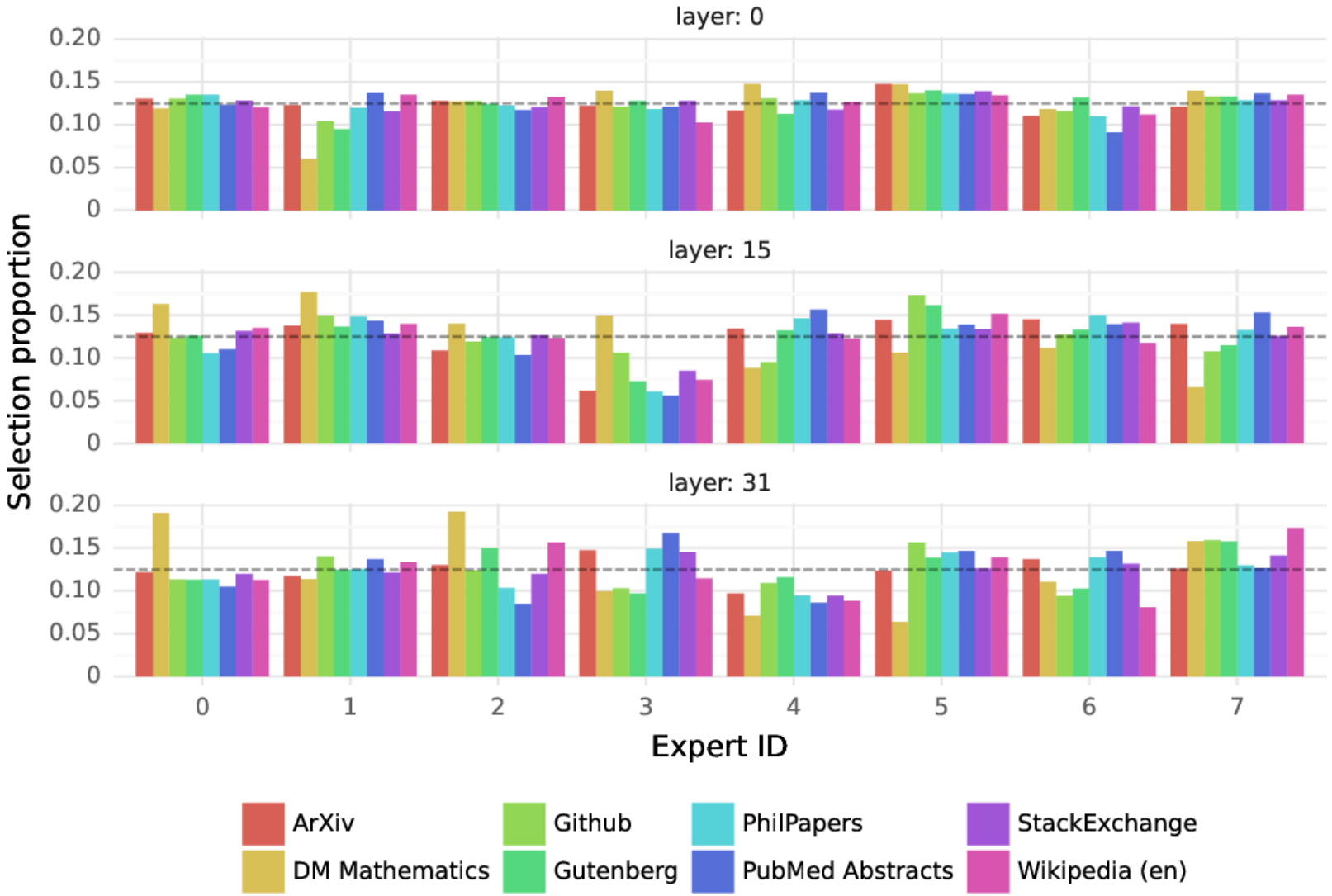
A popular gating function is the softmax function over the top-k logits.

$$G(x) := \text{softmax}(\text{top-}k(x \cdot W_g))$$

In order to have a sparse vector as output

$$\text{top-}k(x \cdot W_g) = \begin{cases} v_i & \text{if } v_i \text{ is in the top } k \text{ of } x \cdot W_g \\ -\infty & \text{otherwise} \end{cases}$$

Mixture of experts



Mixture of experts



Lab Session